

CN7000 - EMNIST Classification

% James Hooper - 2780419

```
%Close all open figures
close all
%Clear the workspace
clear
%Clear the command window
clc
```

```
%Unzip dataset
%unzip('/MATLAB Drive/Dissertation/matlab.zip', '/MATLAB Drive/Dissertation/matlab');
```

```
imds = imageDatastore(fullfile(getenv('USERPROFILE'), 'Documents', 'emnist_letters_png', 'train'), ...
    'IncludeSubfolders', true, ...
    'LabelSource', 'foldernames');
```

```
preview(imds)
```

```
ans = 28x28 uint8 matrix
```

[illegible]

```
img = preview(imds);  
imshow(img);
```



```
countEachLabel(imds)
```

```
ans = 26x2 table
```

	Label	Count
1	A	4800
2	B	4800
3	C	4800
4	D	4800
5	E	4800
6	F	4800
7	G	4800
8	H	4800
9	I	4800
10	J	4800
11	K	4800
12	L	4800
13	M	4800
14	N	4800

...

```
% Verify class balance (EMNIST Letters is balanced by construction)
labelCounts = countEachLabel(imds);
assert(numel(unique(labelCounts.Count)) == 1, ...
    'Class counts unequal - check the conversion completed fully.');
```

```
% Split into training and validation sets
% (test = official EMNIST test partition, reserved for final evaluation only)
rng(42); % fixed seed: identical split every run, and for every model compared later
[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.85, 'randomized');
```

```
imdsTest = imageDatastore(fullfile(getenv('USERPROFILE'), 'Documents', 'emnist_letters_png', 'test'), ...
    'IncludeSubfolders', true, 'LabelSource', 'foldernames');
```

% Per-Class Sample Preview

```
YTrain = imdsTrain.Labels;
classes = categories(YTrain);

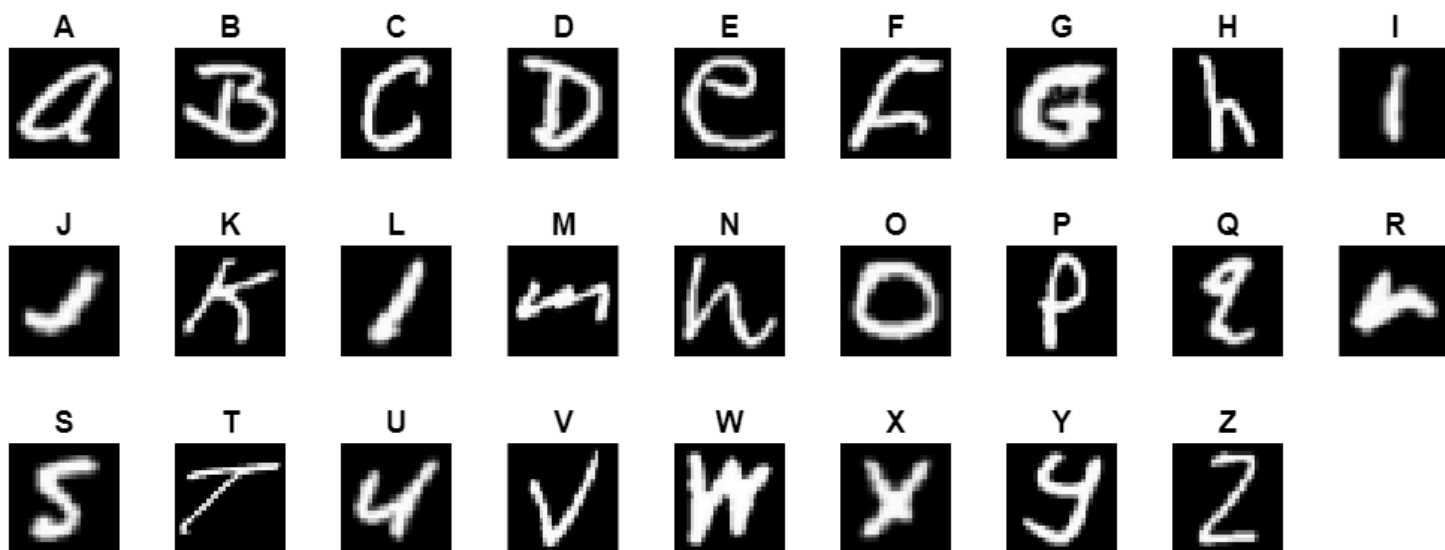
figure('Position', [100, 100, 1400, 600]); % 3x9 = 27 tiles, covers all 26 letters
tiledlayout(3, 9, 'Padding', 'tight', 'TileSpacing', 'compact');

YTrain = imdsTrain.Labels;

for i = 1:numel(classes)
    idx = find(YTrain == classes(i), 1);
    if ~isempty(idx)
        nexttile;
        imshow(readimage(imdsTrain, idx));
        title(string(classes(i)), 'Interpreter', 'none', 'FontSize', 10);
    end
end

sgtitle('Example Image per Class', 'FontSize', 14);
```

Example Image per Class



```
% Data Augmentation (letters-safe: NO reflection - mirroring changes letter identity, e.g. b/d, p/q)
imageAugmenter = imageDataAugmenter( ...
    'RandRotation', [-10 10], ...
    'RandXShear', [-10 10], ... % slant variation - the appropriate substitute for reflection
    'RandXTranslation', [-5 5], ...
    'RandYTranslation', [-5 5]);

augimdsTrain224 = augmentedImageDatastore([224 224], imdsTrain, ...
    'ColorPreprocessing', 'gray2rgb', 'DataAugmentation', imageAugmenter);
augimdsVal224 = augmentedImageDatastore([224 224], imdsValidation, 'ColorPreprocessing', 'gray2rgb');
augimdsTest224 = augmentedImageDatastore([224 224], imdsTest, 'ColorPreprocessing', 'gray2rgb');
```

```
% Define number of images to sample
numImages = 16; % 4x4 grid
perm = randperm(numel(imdsTrain.Files), 16);
sampleImgs = subset(imdsTrain, perm);
augSample = augmentedImageDatastore([64 64], sampleImgs, ...
    'ColorPreprocessing', 'gray2rgb', 'DataAugmentation', imageAugmenter);
batch = readall(augSample);

figure;
montage(batch.input, 'Size', [4 4]);
title('Random Augmented Images Across Classes');
```

```
%checkpoint
save('trainingCheckpoint.mat', 'imds', 'imdsTrain', 'imdsValidation', 'imdsTest', ...
    'augimdsTrain224', 'augimdsVal224', 'augimdsTest224', '-v7.3');
```

%load ResNet18

```
net = resnet18;
lgraph = layerGraph(net);
lgraph = replaceHead(lgraph, numel(classes));

function lgraph = replaceHead(lgraph, numClasses)
    L = lgraph.Layers;
    fcIdx = find(arrayfun(@(l) isa(l, 'nnet.cnn.layer.FullyConnectedLayer'), L), 1, 'last');
    smIdx = find(arrayfun(@(l) isa(l, 'nnet.cnn.layer.SoftmaxLayer'), L), 1, 'last');
    clIdx = find(arrayfun(@(l) isa(l, 'nnet.cnn.layer.ClassificationOutputLayer'), L), 1, 'last');
    lgraph = replaceLayer(lgraph, L(fcIdx).Name, ...
        fullyConnectedLayer(numClasses, 'Name', 'fc_new', 'WeightLearnRateFactor', 10, 'BiasLearnRateFactor', 10));
    lgraph = replaceLayer(lgraph, L(smIdx).Name, softmaxLayer('Name', 'sm_new'));
    lgraph = replaceLayer(lgraph, L(clIdx).Name, classificationLayer('Name', 'out_new'));
end
```

```
analyzeNetwork(lgraph)
```

```
deepNetworkDesigner(lgraph)
```

```
if ~exist('checkpointsResNet18', 'dir')
    mkdir('checkpointsResNet18');
end
```

```
optionsTransfer = trainingOptions('adam', ...
    'InitialLearnRate', 1e-4, ...
    'MaxEpochs', 20, ...
    'MiniBatchSize', 64, ...
    'CheckpointPath', 'checkpointsResNet18', ...
    'Shuffle', 'every-epoch', ...
    'ValidationData', augimdsVal224, ...
    'ValidationFrequency', 400, ...
    'ValidationPatience', 6, ...
    'OutputNetwork', 'best-validation-loss', ...
    'DispatchInBackground', true, ...
    'Verbose', false, ...
    'ExecutionEnvironment', 'gpu', ...
    'Plots', 'training-progress');
```

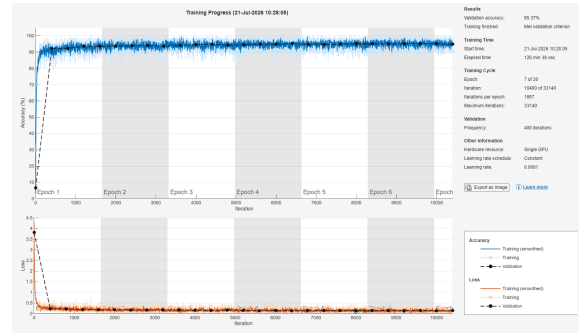
%Train the Network

```
[trainedNet18, info18] = trainNetwork(augimdsTrain224, lgraph, optionsTransfer);
```

Starting parallel pool (parpool) using the 'Processes' profile ...

21-Jul-2026 10:25:48: Job Queued. Waiting for parallel pool job with ID 1 to start ...

Connected to parallel pool with 6 workers.



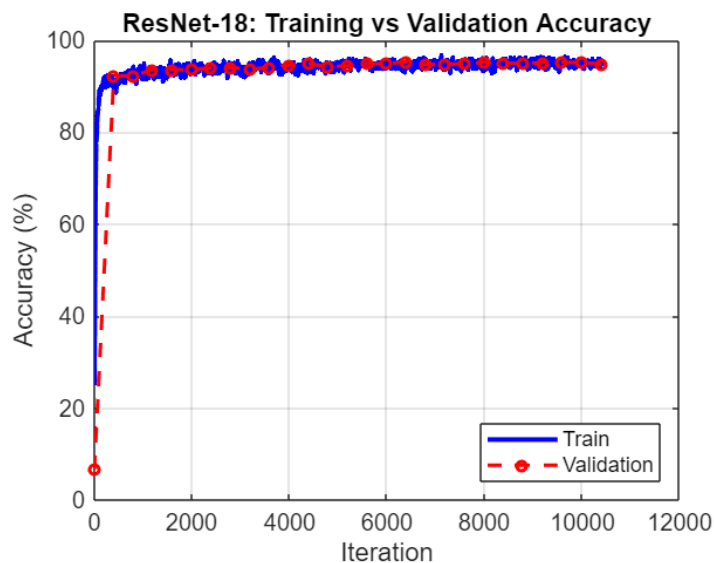
```
save('ResNet18_EMNIST_final.mat', 'trainedNet18', 'info18');
```

% Training log

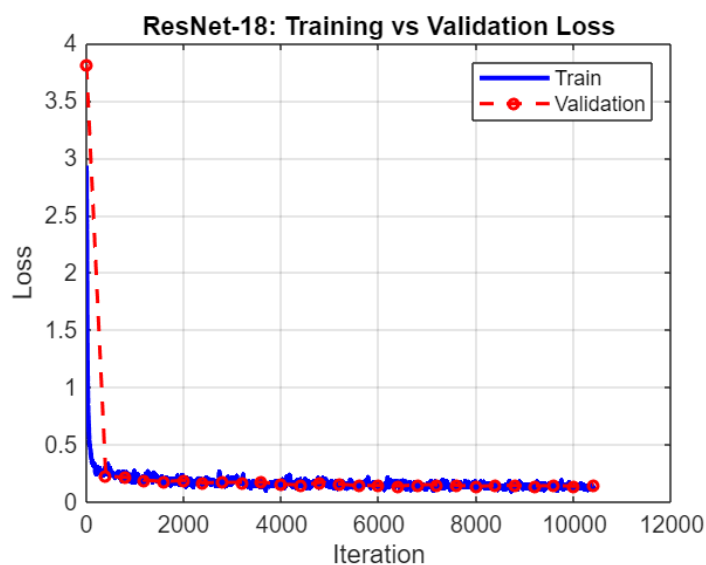
```
logT18 = table((1:numel(info18.TrainingLoss))', info18.TrainingLoss', ...  
    info18.TrainingAccuracy', info18.ValidationLoss', info18.ValidationAccuracy', ...  
    'VariableNames', {'Iteration', 'TrainLoss', 'TrainAccuracy', 'ValLoss', 'ValAccuracy'});  
writetable(logT18, 'TrainingLog_ResNet18_EMNIST.csv');
```

```
T = readtable('TrainingLog_ResNet18_EMNIST.csv');
```

```
figure;  
plot(T.Iteration, smoothdata(T.TrainAccuracy, 'movmean', 20), 'b-', 'LineWidth', 2); hold on;  
v = ~isnan(T.ValAccuracy);  
plot(T.Iteration(v), T.ValAccuracy(v), 'r--o', 'LineWidth', 1.5, 'MarkerSize', 4);  
xlabel('Iteration'); ylabel('Accuracy (%)');  
legend('Train', 'Validation', 'Location', 'southeast');  
title('ResNet-18: Training vs Validation Accuracy'); grid on;  
exportgraphics(gcf, 'fig4_5a_accuracy.png', 'Resolution', 200);
```



```
figure;  
plot(T.Iteration, smoothdata(T.TrainLoss, 'movmean', 20), 'b-', 'LineWidth', 2); hold on;  
w = ~isnan(T.ValLoss);  
plot(T.Iteration(w), T.ValLoss(w), 'r--o', 'LineWidth', 1.5, 'MarkerSize', 4);  
xlabel('Iteration'); ylabel('Loss');  
legend('Train', 'Validation', 'Location', 'northeast');  
title('ResNet-18: Training vs Validation Loss'); grid on;  
exportgraphics(gcf, 'fig4_5b_loss.png', 'Resolution', 200);
```

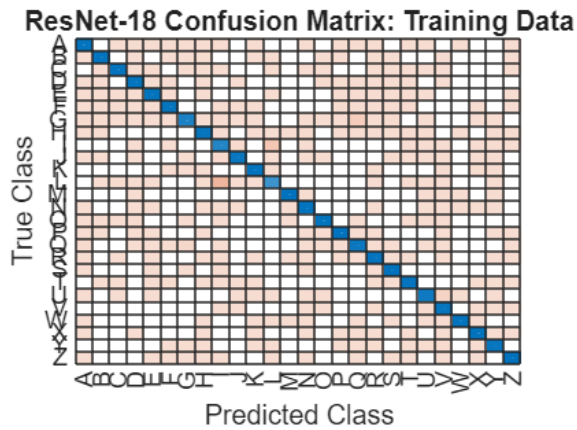


% Confusion Matrix and Predictions

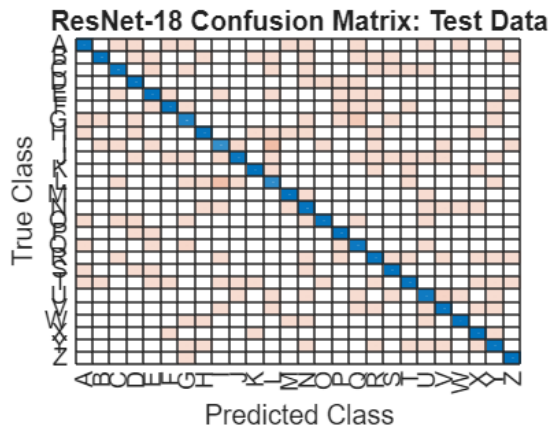
```
YTrueTrain = imdsTrain.Labels;  
YTrueTest  = imdsTest.Labels;  
  
YPredTrain = classify(trainedNet18, augimdsTrain224);  
YPredTest  = classify(trainedNet18, augimdsTest224);  
  
trainAccuracy = mean(YPredTrain == YTrueTrain);  
testAccuracy  = mean(YPredTest == YTrueTest);  
  
fprintf('Training Accuracy: %.2f%%\n', trainAccuracy * 100);
```

Training Accuracy: 95.67%

```
figure;  
confusionchart(YTrueTrain, YPredTrain);  
title('ResNet-18 Confusion Matrix: Training Data');
```



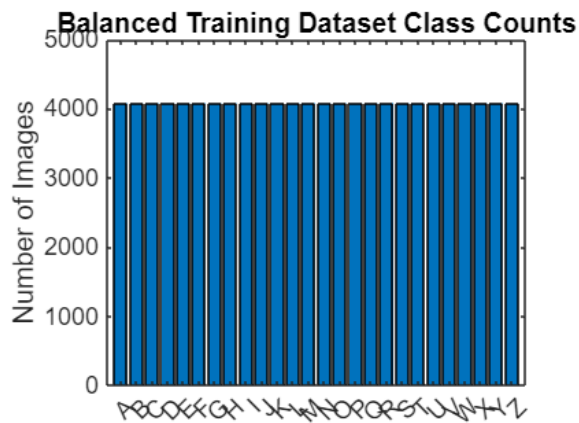
```
figure;  
confusionchart(YTrueTest, YPredTest);  
title('ResNet-18 Confusion Matrix: Test Data');
```



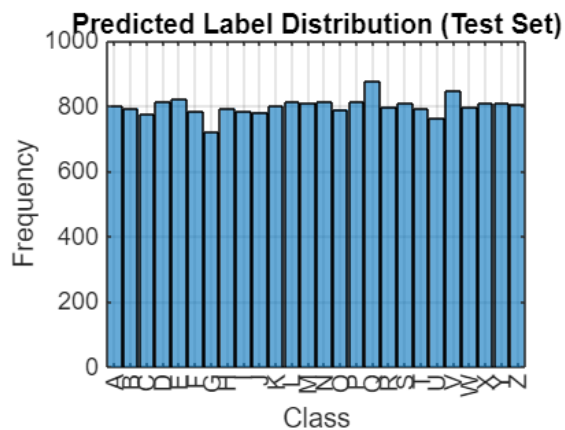
```
fprintf('Test Accuracy: %.2f%%\n', testAccuracy * 100);
```

Test Accuracy: 95.26%

```
% Dataset balancing chart
figure;
labelCounts = countEachLabel(imdsTrain); % or imdsBalanced
bar(labelCounts.Label, labelCounts.Count);
xtickangle(45);
ylabel('Number of Images');
title('Balanced Training Dataset Class Counts');
```



```
% Histogram of Predictions
figure;
histogram(YPredTest);
title('Predicted Label Distribution (Test Set)');
xlabel('Class');
ylabel('Frequency');
grid on;
```

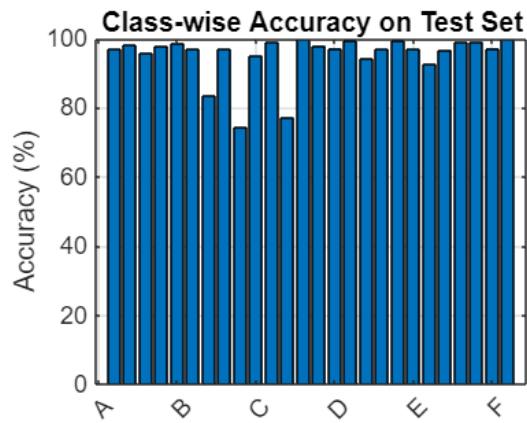


```

% Class-wise Accuracy (Test Set)
classes = categories(YTrueTest);
accPerClass = zeros(numel(classes),1);
for i = 1:numel(classes)
    idxClass = (YTrueTest == classes{i});
    accPerClass(i) = mean(YPredTest(idxClass) == YTrueTest(idxClass));
end

figure;
bar(accPerClass * 100);
set(gca, 'XTickLabel', classes, 'XTickLabelRotation', 45);
ylabel('Accuracy (%)');
title('Class-wise Accuracy on Test Set');
grid on;

```




```
% F1 Score Calculation (Test Set)
f1s = zeros(numel(classes), 1);
for i = 1:numel(classes)
    tp = sum((YPredTest == classes(i)) & (YTrueTest == classes(i)));
    fp = sum((YPredTest == classes(i)) & (YTrueTest ~= classes(i)));
    fn = sum((YPredTest ~= classes(i)) & (YTrueTest == classes(i)));
    precision = tp / (tp + fp + eps);
    recall = tp / (tp + fn + eps);
    f1s(i) = 2 * (precision * recall) / (precision + recall + eps);
end
macroF1_ResNet18 = mean(f1s);
fprintf('Macro-F1: %.4f\n', macroF1_ResNet18);
```

Macro-F1: 0.9526

```
% print sorted scores
[sortedF1, sortIdx] = sort(f1s, 'descend');
sortedClasses = classes(sortIdx);

fprintf('F1 Scores (sorted):\n');
```

F1 Scores (sorted):

```
for i = 1:numel(sortedF1)
    fprintf('%-12s : %.3f\n', string(sortedClasses(i)), sortedF1(i));
end
```

Z : 0.994
M : 0.993
W : 0.991
K : 0.991
S : 0.988
X : 0.986
B : 0.986
P : 0.985
F : 0.980
O : 0.979
T : 0.975
H : 0.974
E : 0.974
C : 0.973
N : 0.972
D : 0.971
R : 0.971
A : 0.969
Y : 0.968
J : 0.963
U : 0.948
V : 0.941
Q : 0.899

```
[YPredTest, ~] = classify(trainedNet18, augimdsTest224);
YTrueTest = imdsTest.Labels;

predT = table(imdsTest.Files, YTrueTest, YPredTest, ...
    'VariableNames', {'ImagePath', 'TrueLabel', 'PredLabel'});
writetable(predT, 'Predictions_ResNet18_EMNIST.csv');
```

ans = 1x3 cell

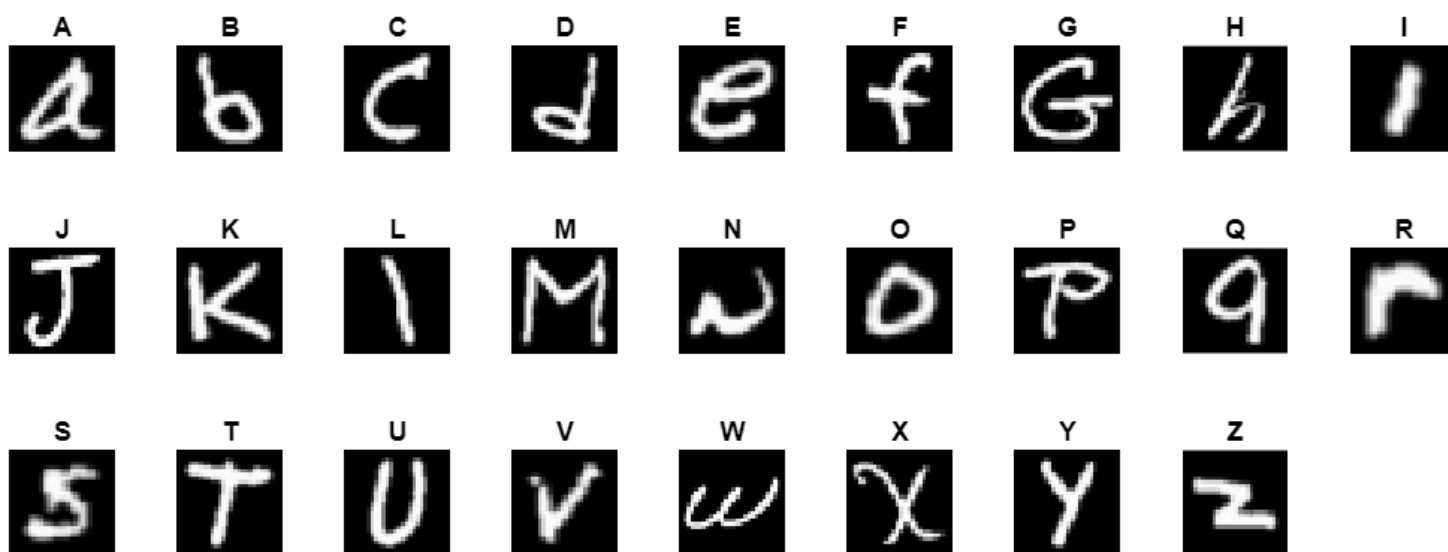
'ImagePath' 'TrueLabel' 'PredLabel'

ImagePath	TrueLabel	PredLabel
C:\Users\James\Documents\emnist_letters_png\test\A\000001.png	"A"	"A"
C:\Users\James\Documents\emnist_letters_png\test\A\000002.png	"A"	"A"
C:\Users\James\Documents\emnist_letters_png\test\A\000003.png	"A"	"A"
C:\Users\James\Documents\emnist_letters_png\test\A\000004.png	"A"	"A"
C:\Users\James\Documents\emnist_letters_png\test\A\000005.png	"A"	"A"
C:\Users\James\Documents\emnist_letters_png\test\A\000006.png	"A"	"A"
C:\Users\James\Documents\emnist_letters_png\test\A\000007.png	"A"	"A"
C:\Users\James\Documents\emnist_letters_png\test\A\000008.png	"A"	"A"

```
% Rebuild everything needed from disk – no retraining, no reclassifying
predT = readtable('Predictions_ResNet18_EMNIST.csv', opts);
imdsTest = imageDatastore(predT.ImagePath, 'Labels', categorical(predT.TrueLabel));
YTrueTest = categorical(predT.TrueLabel);
YPredTest = categorical(predT.PredLabel);
classes = categories(YTrueTest);

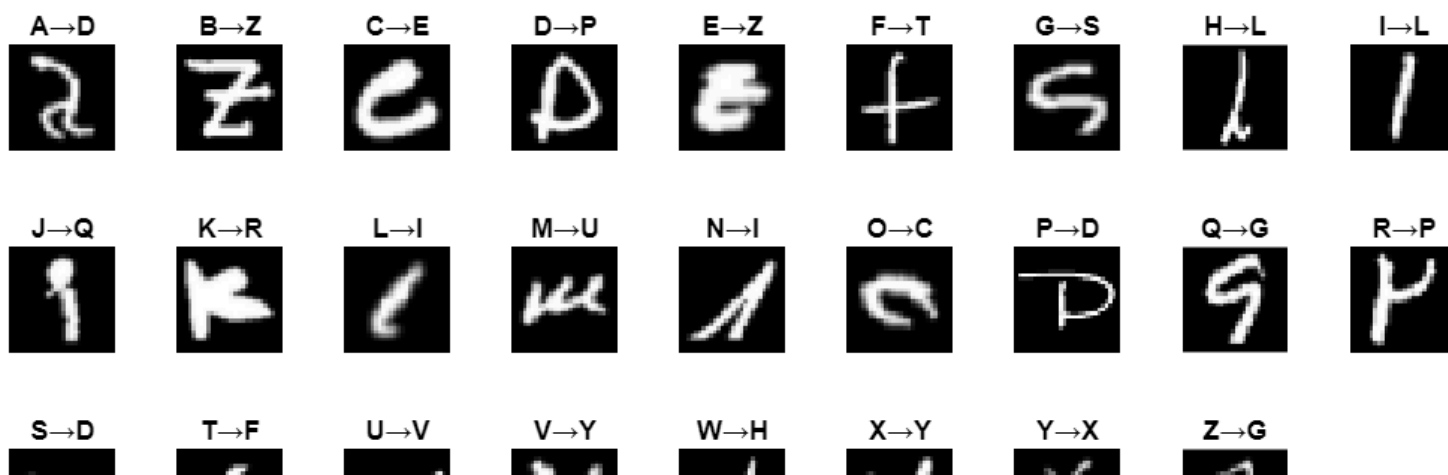
% Correctly classified
figure('Position', [100, 100, 1400, 600]);
tiledlayout(3, 9, 'Padding', 'tight', 'TileSpacing', 'normal')
for i = 1:numel(classes)
    idx = find((YPredTest == YTrueTest) & (YTrueTest == classes(i)));
    if ~isempty(idx)
        randIdx = idx(randi(numel(idx)));
        nexttile;
        imshow(imread(predT.ImagePath{randIdx}));
        title(string(classes(i)), 'Interpreter', 'none', 'FontSize', 9);
    end
end
sgtitle('One Correct Prediction per Class', 'FontSize', 14);
exportgraphics(gcf, 'fig4_6a_correct.png', 'Resolution', 200);
```

One Correct Prediction per Class



```
% Misclassified
figure('Position', [100, 100, 1400, 600]);
tiledlayout(3, 9, 'Padding', 'tight', 'TileSpacing', 'normal')
for i = 1:numel(classes)
    idx = find((YPredTest ~= YTrueTest) & (YTrueTest == classes(i)));
    if ~isempty(idx)
        randIdx = idx(randi(numel(idx)));
        nexttile;
        imshow(imread(predT.ImagePath{randIdx}));
        title(string(YTrueTest(randIdx)) + "→" + string(YPredTest(randIdx)), ...
            'Interpreter', 'none', 'FontSize', 9);
    end
end
sgtitle('One Misclassified Prediction per Class', 'FontSize', 14);
exportgraphics(gcf, 'fig4_6b_misclassified.png', 'Resolution', 200);
```

One Misclassified Prediction per Class





```
ans = 20800x1 categorical
```

```
A  
A  
A  
A  
A  
A  
A  
A  
A  
A  
A
```

```
⋮
```

```
% Save the trained network for future inference  
save('ResNet18_EMNIST_final.mat', 'trainedNet18', 'info18', '-v7.3');
```